

Aspectos gerais do reuso

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução ao Reuso de Software
- Aspectos Gerais do Reuso
- Níveis de Reuso
- Tipos de Artefatos Reutilizáveis
- Prática – Refatoração para Reuso
 - Problema Inicial (código repetido)
 - Identificação da Redundância
 - Refatoração em Módulo Utilitário
 - Reuso em outro projeto

Introdução ao Reuso de Software

- Definição segundo IEEE 1517: “Uso de ativos existentes em processos de desenvolvimento de software com o propósito de construir novos produtos de software.”
- Um ativo de software pode ser código, documentação, requisitos, testes, arquiteturas, modelos ou até conhecimento organizacional.

Porque reusar?

- **Redução de custos:** desenvolvimento mais barato porque parte do trabalho já está pronto.
- **Redução do tempo de entrega:** acelera a produção de novas funcionalidades.
- **Aumento da qualidade:** componentes já testados e validados reduzem bugs.
- **Consistência:** padronização de soluções em toda a organização.
- **Inovação com base sólida:** tempo livre para focar em diferenciais, não no "básico".

Porque reusar?

- Estudo clássico (Krueger, 1992): 40–60% de um sistema típico pode ser construído a partir de software existente.
- Empresas líderes (Google, Microsoft, Amazon) usam repositórios internos de componentes, frameworks e serviços que sustentam centenas de produtos.



Tipos e níveis de reuso

- **Reuso ad hoc** – copiar e colar código (baixo nível, pouco controlado).
- **Reuso sistemático** – uso de bibliotecas, frameworks e APIs.
- **Reuso organizacional** – componentes certificados, mantidos em repositórios corporativos.
- **Reuso em larga escala** – arquiteturas de referência, plataformas de produto (ex.: Android, AWS)

Exemplos de reuso no cotidiano

- **Frameworks front-end:** React, Angular, Vue.
- **APIs:** Google Maps, Firebase, Stripe, OpenAI.
- **Templates de projetos:** Spring Initializr, Angular CLI.
- **Bibliotecas de Machine Learning:** TensorFlow, PyTorch, Scikit-Learn.
- **Infraestrutura:** Docker, Kubernetes, Terraform (infra as code reutilizável).
- **Componentes de design:** Material UI, Tailwind, Bootstrap.

Estratégias de reuso

- **Componentização:** dividir o software em módulos independentes.
- **Serviços:** SOA e microserviços reutilizáveis em diferentes aplicações.
- **Product Line Engineering (LPS):** desenvolvimento baseado em plataformas de produtos.
- **Reuso orientado a domínio:** criar bibliotecas para problemas recorrentes no domínio da empresa.
- **Open Source:** reuso de componentes da comunidade (ex.: Linux, PostgreSQL, Redis).

Boas práticas de reuso

- Criar documentação clara dos componentes.
- Usar controle de versão (GitHub, GitLab, Bitbucket).
- Definir contratos de interface estáveis (ex.: OpenAPI/Swagger).
- Adotar testes automatizados para garantir confiabilidade.
- Manter um repositório corporativo com busca e classificação de ativos.
- Estimular cultura de compartilhamento interno entre equipes.

Casos práticos de reuso

- **Google:** todos os produtos compartilham bibliotecas internas (monorepo).
- **Netflix:** OSS reusado por várias empresas (Hystrix, Zuul, Eureka).
- **Amazon:** APIs internas viraram produtos AWS (ex.: S3, DynamoDB).
- **Startups:** aceleram lançamentos usando frameworks + APIs terceiras.

Aspectos Gerais do Reuso

- Reuso de software é a prática de aproveitar ativos já existentes no processo de desenvolvimento de novos sistemas.
- Esses ativos podem ser código, bibliotecas, APIs, frameworks, especificações, modelos, testes ou até documentação.
- O objetivo é evitar reinventar a roda, reduzindo custo, tempo e risco no desenvolvimento.

Benefícios gerais

- Economia de tempo e custo.
- Qualidade e confiabilidade (componentes já testados).
- Padronização entre diferentes projetos.
- Facilidade de manutenção (menos código duplicado).
- Aceleração da inovação (foco no que é novo e específico do produto).

Desafios gerais

- Compatibilidade técnica entre versões, linguagens e plataformas.
- **Adaptação:** nem sempre o componente atende 100% da nova necessidade.
- **Dependências:** riscos de acoplamento excessivo.
- **Gerenciamento de ativos:** documentação, versionamento, catálogo.
- **Questões legais:** licenciamento de software open source ou comercial.
- Custo inicial para criar componentes realmente reutilizáveis.

Processos de Reuso

- IEEE 1517 / ISO 12207: definem ciclo de vida de reuso.
- **Processo típico:** identificação → adaptação → integração → manutenção.
- **Ferramentas de apoio:**
 - Catálogos de componentes.
 - Gerenciadores de pacotes (npm, Maven, pip).
 - Repositórios corporativos de APIs.

Relação com Engenharia de Software

- O reuso não é só um detalhe técnico, mas um paradigma estratégico:
- Se conecta à Engenharia de Domínio (construir ativos genéricos reutilizáveis).
- Apoia LPS (Linha de Produtos de Software).
- Sustenta DevOps e Arquitetura em Nuvem (infraestrutura como código, serviços serverless).

Níveis de Reuso

- Ad hoc → improviso.
- Bibliotecas → organizado.
- Componentes → encapsulado.
- Frameworks → estruturado.
- Serviços/APIs → distribuído.
- Plataformas → ecossistemas.
- LPS → engenharia de produto em larga escala.

Reuso de código

- **Definição:** aproveitamento direto de funções, classes ou bibliotecas que encapsulam lógica reutilizável.
- **Exemplos:**
 - hashlib (Python) para criptografia e hashing.
 - java.util (Java) para coleções e utilitários.
 - Pacotes NPM como lodash (JavaScript).
- **Observação:** nível mais básico e mais difundido, mas exige padronização para evitar “colcha de retalhos”.

Reuso de Componentes

- **Definição:** encapsulam funcionalidades independentes, com interfaces bem definidas, podendo ser plugados em diferentes aplicações.
- **Exemplos:**
 - Angular: componentes de UI reutilizáveis.
 - Flutter: Widgets para interfaces multiplataforma.
 - .NET: componentes de interface e acesso a dados.
- **Observação:** reduz duplicação, promove modularidade e isolamento de responsabilidades.

Reuso de Frameworks

- **Definição:** fornecem estruturas arquiteturais + padrões de desenvolvimento, servindo de base para aplicações em um domínio específico.
- **Exemplos:**
 - Django (Python) – aplicações web.
 - Spring (Java) – aplicações corporativas e microserviços.
 - Ruby on Rails – aplicações web rápidas e seguras.
- **Observação:** frameworks guiam o desenvolvimento, mas também podem impor restrições (lock-in tecnológico).

Reuso de Serviços / APIs

- **Definição:** aproveitamento de serviços internos ou externos, via protocolos e contratos de integração.
- **Exemplos:**
 - Stripe, PayPal – serviços de pagamento.
 - Google Maps API – mapas e geolocalização.
 - Firebase Authentication – autenticação e login social.
- **Observação:** central em arquiteturas baseadas em APIs e microserviços, viabiliza escalabilidade e integração entre sistemas.

Reuso Arquitetural

- **Definição:** aplicação de padrões arquiteturais e estilos de desenvolvimento para construir famílias de sistemas.
- **Exemplos:**
 - MVC (Model-View-Controller) – separação de responsabilidades em aplicações web.
 - Microserviços – sistemas distribuídos independentes.
 - Event-Driven Architecture – processamento baseado em eventos.
- **Observação:** aumenta a consistência organizacional, facilita manutenção e evolução.

Reuso em Linhas de Produto de Software (SPL)

- **Definição:** construção de famílias de produtos que compartilham ativos comuns, com variabilidade planejada.
- **Exemplos:**
 - Microsoft Office (Word, Excel, PowerPoint compartilham componentes internos).
 - Sistemas automotivos (infotainment, telemetria, controle embarcado).
- **Observação:** exige engenharia de domínio e infraestrutura de gestão de variabilidade.

Reuso de Conhecimento e Processos

- **Definição:** reutilização de artefatos não funcionais (documentos, processos, templates, práticas).
- **Exemplos:**
 - Documentação de requisitos reutilizável.
 - Diagramas UML aplicáveis a múltiplos projetos.
 - Templates de pipelines CI/CD em YAML (DevOps).
 - Casos de teste automatizados reaproveitados.
- **Observação:** fundamental para maturidade organizacional, não só técnica.

Quadro Comparativo

Nível de Reuso	O que é reutilizado	Exemplo
Código	Funções, classes, bibliotecas	hashlib, lodash
Componentes	Funcionalidades encapsuladas	Widgets Flutter, componentes Angular
Frameworks	Estruturas arquiteturais + padrões	Django, Spring
Serviços / APIs	Serviços internos/externos via integração	Stripe, Google Maps API
Arquitetural	Padrões de arquitetura	MVC, Microserviços
Linhas de Produto (SPL)	Famílias de sistemas com ativos compartilhados	Microsoft Office, sistemas automotivos
Conhecimento e Processos	Documentos, modelos, templates, testes	Pipelines CI/CD, modelos UML

Discussão

- Qual nível de reuso sua experiência mostra ser mais comum na prática (acadêmica, industrial, pessoal)?
- Por que empresas ainda reinventam componentes?
- Como equilibrar inovação e padronização?
- O que é mais difícil: reusar bem ou fazer algo que será reutilizável?

Tipos de Artefatos Reutilizáveis

- Artefatos de Código
 - Funções, classes, pacotes, bibliotecas.
 - Exemplo: módulo math em Python, java.util em Java.
- Componentes de Software
 - Binários ou módulos distribuídos (.jar, .dll, pacotes npm).
 - Exemplo: pacotes de autenticação.

Tipos de Artefatos Reutilizáveis

- Frameworks
 - Fornecem arcabouço: definem como aplicações devem ser estruturadas.
 - Exemplo: Android SDK.
- Serviços e APIs
 - Artefatos de reuso em nível de rede.
 - REST, GraphQL, gRPC.
 - Exemplo: Firebase Auth, AWS S3.

Tipos de Artefatos Reutilizáveis

- Modelos e Especificações
 - Modelos UML, DSLs, templates de requisitos.
 - Exemplo: caso de uso padrão de “Login de usuário”.
- Padrões Arquiteturais e de Projeto
 - GoF (Strategy, Observer, Factory).
 - Arquiteturas de referência (SOA, microserviços).

Tipos de Artefatos Reutilizáveis

- Artefatos de Teste
 - Suítes automatizadas (JUnit, PyTest).
 - Mocks, stubs, scripts de integração.
- Infraestrutura e Configuração
 - Templates de Docker, Kubernetes YAML, scripts Ansible.
 - Exemplo: pipeline de CI/CD reutilizado em múltiplos projetos.
- Documentação
 - Guias de estilo, templates de design doc.
 - Checklists de revisão de código.

Prática – Refatoração para Reuso

- Objetivo da prática
 - remover duplicação (“don’t repeat yourself – DRY”)
 - extrair funções utilitárias puras (sem efeitos colaterais)
 - organizar em um módulo comum com nomes claros, testes e documentação curta
 - deixar o caminho pronto para empacotar (lib interna) ou expor como API

Problema

- Funções quase idênticas.
- Lógica repetida (cálculo sobre `produto["preco"]`).
- Difícil de manter e estender (se mudar a regra, tem que alterar em vários lugares).

Prática – Refatoração para Reuso

```
def preco_com_desconto(produto, desconto):
    return produto["preco"] - (produto["preco"] * desconto)

def preco_com_imposto(produto, imposto):
    return produto["preco"] + (produto["preco"] * imposto)
```

Identificação da Redundância

- Padrão comum: aplicar um fator multiplicador sobre o preço.
- Variação: desconto $\rightarrow$ fator < 1 | imposto $\rightarrow$ fator > 1 .

Refatoração em Módulo Utilitário

```
# utils/precos.py

def aplicar_fator(preco: float, fator: float) -> float:
    return preco * fator

def preco_com_desconto(preco: float, desconto: float) -> float:
    return aplicar_fator(preco, 1 - desconto)

def preco_com_imposto(preco: float, imposto: float) -> float:
    return aplicar_fator(preco, 1 + imposto)
```

Refatoração em Módulo Utilitário

- Vantagens:
 - Centralização da lógica comum.
 - Funções reutilizáveis, coesas e fáceis de testar.
 - Redução de código duplicado → manutenção simplificada.

Reuso em Outro Projeto

```
from utils.precos import preco_com_desconto, preco_com_imposto

print(preco_com_desconto(100, 0.1))    # Saída: 90.0
print(preco_com_imposto(100, 0.2))    # Saída: 120.0
```

- Agora o módulo `utils` pode ser importado em diferentes projetos.

Extensões Possíveis

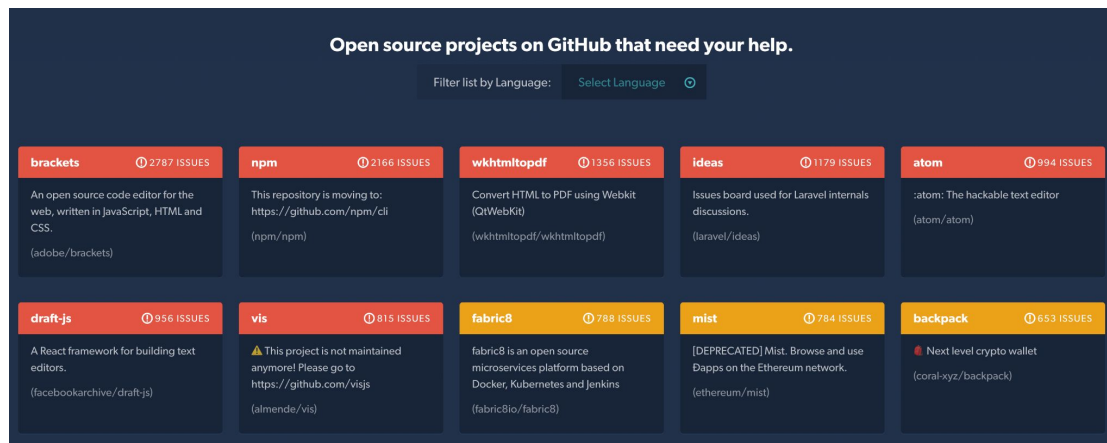
- Empacotamento: transformar `utils/precos.py` em um pacote PyPI.
- Serviço: expor como uma API REST (FastAPI ou Flask).
- Documentação: gerar docstrings e usar ferramentas como Sphinx ou MkDocs.
- Versionamento: versionar no Git e adotar versionamento semântico (ex.: v1.0.0).

Discussão

- O que se ganhou com a refatoração em termos de qualidade, manutenção e reuso?
- Esse módulo pode evoluir para algo compartilhado entre projetos (PyPI ou serviço REST)?
- Quais cuidados devem ser tomados ao documentar, versionar e distribuir utilitários como esse?
- Como essa prática se conecta aos níveis de reuso (código → componente → serviço)?

Escolha de projeto legado

- Cada dupla deve procurar um projeto “legado base” para refatorar ao longo do semestre.



<https://www.codetriage.com/>

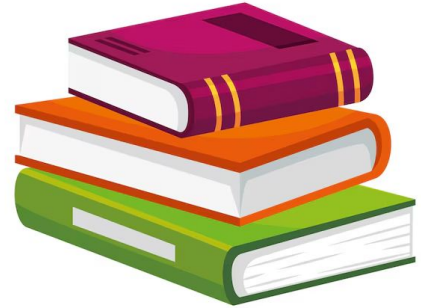
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

